

Exercícios de fixação

Instruções gerais

Vocês terão até o final da aula para resolver todas as questões propostas. Cada problema deverá ser implementado em um arquivo separado, utilizando os conceitos estudados durante o minicurso. Durante a realização das atividades, vocês poderão consultar os materiais disponibilizados e tirar dúvidas com os petianos presentes. Procurem ler atentamente os enunciados, planejar a solução antes de começar a programar e testar o código para verificar se ele está funcionando corretamente.

Essa lista de exercícios não tem um caráter avaliativo, o objetivo é praticar o que foi visto hoje em sala de aula, prepará-los e gerar familiaridade com as ferramentas necessárias para o bom entendimento dos assuntos seguintes.

Problema A: Cadastro de Alunos

Uma escola deseja armazenar informações básicas sobre seus alunos e identificar se cada estudante foi aprovado ou reprovado.

Para isso, crie uma classe ou struct chamada **Aluno**. Essa classe deve possuir dois atributos: o nome do aluno, armazenado em uma **string**, e a nota final, armazenada em uma variável do tipo **float**.

Além disso, implemente um método chamado **situacao()**, que deve informar se o aluno foi aprovado ou reprovado. Considere que um aluno é aprovado quando sua nota é maior ou igual a **7,0**. Caso contrário, ele deve ser considerado reprovado.

obs: Lembrando que se você optar pelo uso de classes, será necessário declarar os atributos e métodos como **public**.

Exemplo

Nome: Ana

Nota: 8.5

Situacao: Aprovado

Requisito

A verificação da situação do aluno deve ser realizada por um método da classe. Evite colocar essa lógica diretamente na função **main()**.

Problema B: Sequência de Fibonacci Recursiva

A sequência de Fibonacci é uma sequência numérica em que cada termo, a partir do terceiro, é obtido pela soma dos dois termos anteriores. Os primeiros valores da sequência são:

0, 1, 1, 2, 3, 5, 8, 13, 21, ...

Por exemplo:

$$F(0) = 0$$

$$F(1) = 1$$

$$F(2) = F(1) + F(0) = 1$$

$$F(3) = F(2) + F(1) = 2$$

$$F(4) = F(3) + F(2) = 3$$

Crie uma função recursiva chamada:

int fibonacci(int n);

A função deve receber um número inteiro **n** e retornar o termo que ocupa a posição **n** na sequência de Fibonacci.

Para resolver o problema, considere a seguinte relação:

$$F(n) = F(n - 1) + F(n - 2)$$

Entrada

O programa deve receber um número inteiro **n**, representando a posição desejada na sequência.

Saída

O programa deve mostrar o valor correspondente à posição **n** da sequência de Fibonacci.

Exemplo 1

Entrada:

6

Saída:

8

Exemplo 2

Entrada:

10

Saída:

55

Requisitos

- A solução deve utilizar uma função recursiva;
- Não utilize estruturas de repetição, como for, while ou do while;
- Considere valores no intervalo:

$$0 \leq n \leq 40$$

Problema C: Analisador de Texto

Uma `std::string` pode ser utilizada como um contêiner de caracteres. Isso significa que é possível acessar cada caractere individualmente, verificar o tamanho do texto e utilizar diferentes operações oferecidas pela própria classe.

Crie um programa que leia uma frase completa e apresente as seguintes informações:

- A quantidade total de caracteres da frase;
- A quantidade de vogais presentes;
- O primeiro caractere;
- O último caractere.

Exemplo

Entrada:

Ola mundo

Saída:

Quantidade de caracteres: 9

Quantidade de vogais: 4

Primeiro caractere: O

Ultimo caractere: o

Requisito

Utilize recursos da classe `std::string`, como:

```
texto.size();  
texto[i];  
texto.front();  
texto.back();
```

Problema D: Maior Elemento de um Array

Um sistema precisa analisar uma sequência fixa de valores e identificar qual deles é o maior.

Crie um programa utilizando:

```
std::array<int, 8>
```

O programa deve ler oito números inteiros e armazená-los no **array**. Após a leitura, percorra os elementos e determine:

- O maior valor armazenado;
- A posição em que esse valor aparece.

Exemplo

Entrada:

4 9 2 15 7 1 8 3

Saída:

Maior valor: 15

Posicao: 3

Requisito

Utilize a biblioteca:

```
#include <array>
```

e declare o contêiner da seguinte forma:

```
std::array<int, 8> numeros;
```

adicional

Se estiver com tempo, trate a possibilidade de o maior elemento ocorrer mais de uma vez.

Problema E — Soma Utilizando Iteradores

Iteradores são objetos utilizados para percorrer os elementos de um contêiner. Eles funcionam de maneira semelhante a ponteiros e permitem acessar os valores armazenados em estruturas como `std::array` e `std::vector`.

Crie um programa que utilize:

```
std::array<int, 10>
```

O programa deve ler dez números inteiros e calcular a soma de todos os valores armazenados. Entretanto, o percurso do array deve ser realizado exclusivamente por meio de iteradores.

Exemplo

Entrada:

```
1 2 3 4 5 6 7 8 9 10
```

Saída:

```
Soma: 55
```

Requisito

Utilize um iterador para percorrer o array, seguindo uma estrutura semelhante à apresentada abaixo:

```
std::array<int, 10>::iterator it;
```

```
for (it = numeros.begin(); it != numeros.end(); it++) {
```

```
    //Lembrando que como iteradores são ponteiros, será necessário desreferenciar para  
    acessar o valor “ *it ”.
```

```
}
```